CISC 5790 Data Mining Final Project Report

Contributors: [redacted], Paige Mitchell

Background

Over the past decade, national economic trends have been less than predictable for the average American. Moreover, from the immediate pitfalls many have had to endure, result from a general decline in employment wages with an incline in daily expenditures. Americans more than ever must ensure their own financial stability. As the market strays away from the favor of the working class, our group strives to classify income values based on various factors that can provide actionable insights and allow working-class Americans the knowledge to make educated decisions to ensure long-term financial success. Utilizing upwards of 32,000 records documenting attributes including but not limited to occupation, education, and age – we deploy the various concepts and theorems introduced throughout this course to further our contribution to this subject matter.

Our selected algorithms include Logistic Regression, Decision Trees, and K-nearest Neighbors (KNN) –respectively all classification models. Our approach involves sufficient preprocessing in order to have these 3 learners culminate into an Ensemble, which will improve our classification accuracy. In doing so, we aim to create the bare bones of what can become a more user-friendly model – becoming an eligible solution to financial and socioeconomic education.

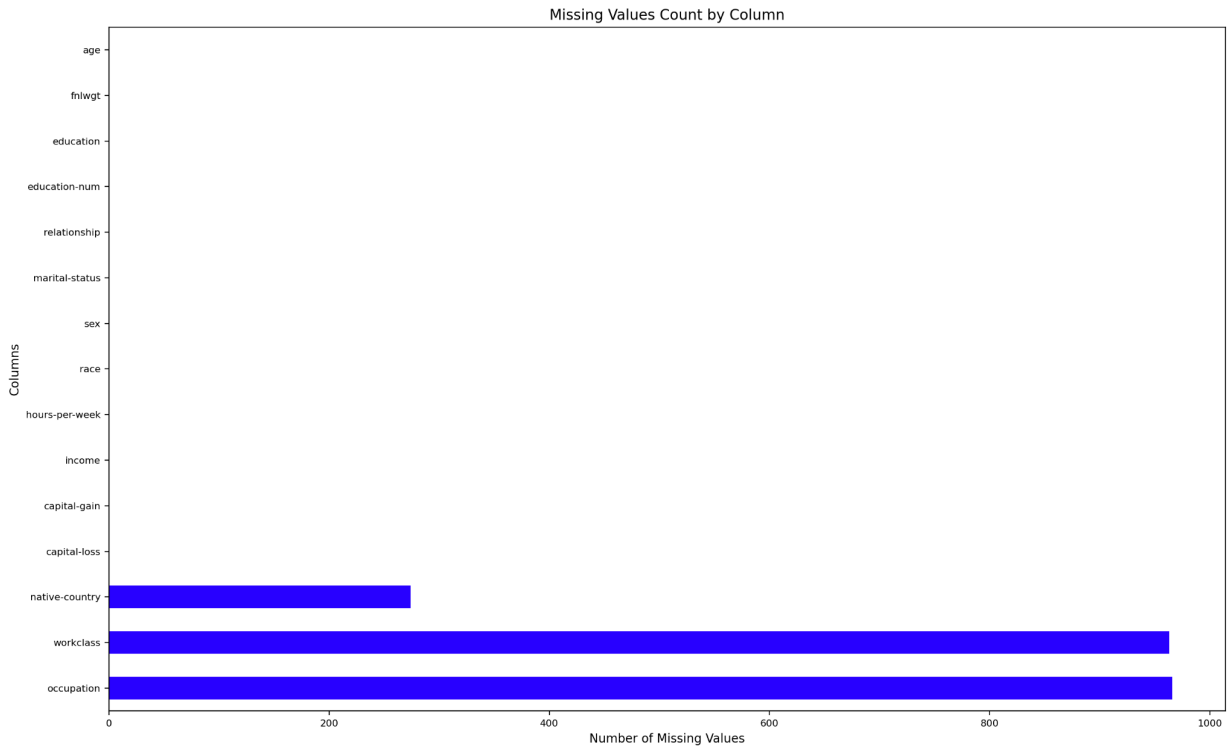
Data Cleaning and Preprocessing

As mentioned in the introduction, the goal of this analysis is to better understand how to make better long-term decisions based on a set of data pulled from the American population. The compilation of data that was used comprised more than 32,000 American adults surveyed (18 and older), accounting for qualities such as sex, race, education level, marital status and even amount of capital gains acquired. As those attributes are considered (namely being placed under variable X_1, X_2,...X_n), they aim to predict the income level into two categories: less than or equal to $50,000 or greater than $50,000 in yearly income. Before pulling non-trivial insights from this data, the following steps were taken to have a clean data set to work with.

Missing Data Imputation

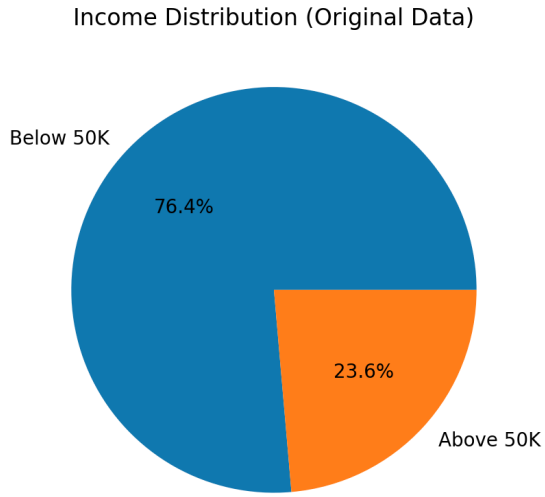
While prepping the data, there were some red flags raised about the totality of the data. For the attributes “workclass”, “occupation”, and “native country”, they were missing data at rates of 5.64%, 5.66%, and 1.79% respectively. The general rule of thumb postulates that if an attribute is missing more than 30% of its values, it is best practice to “drop off the column” or to rid the data set of that attribute entirely. In this case, the rates were not that high, leading to the use of data imputation to continue with the use of these attributes. For data imputation, there are two common ways to execute it: using the mode (the values that repeat most within given

parameters) or kNN imputation, which is ultimately more practical from data science and analysis perspective. From an analytical perspective, kNN is better able to handle multivariate data (maintaining relationship between all attributes), typically more accurate because it functions off of similar data points (k-neighbors), and is also more localized (meaning less chance for bias). Using this imputation method, which finds K of 5 nearest neighbors, it uses Euclidian distance to ensure similar features between the missing and the already existing.



Balancing data

The next step is data balancing, which ensures that the distribution of classes (attributes) are roughly equal so that the model does not create a bias towards one or more attributes over others. We considered two ways to balance the data set: oversampling or undersampling. Oversampling increases the samples of the minority class, usually by duplicating the existing values, such that the model has more data to learn the patterns of the minority class. Undersampling is the direct opposite of that, where the majority class is minimized, such that it doesn't have an overdominating effect on machine learning. Considering that the current split of the classes are ≤$50K~75% and >$50K~25%, the best course of action was to pursue oversampling as the method for balancing the data. After the balance, the resample is split evenly down the middle, with both classes containing 24,720 data points. If undersampling were used, it would be even, but there would be 7,841 data points per class, significantly reducing the amount of information density.



Normalization

The final step towards cleansing the data is normalization, which ensures that all values and attributes are operating on an equal playing field. A computer is able to recognize the difference in magnitude between values, so for example a zip code (a 5 digit number) will be weighed much higher than how many pets one may have (usually single digit). However, the process for normalizing continuous vs. discrete/categorical data varies. For continuous variables, there are two approaches: the min-max method and the z-score method. Min-max simply places all continuous values within the range of the min-max of said category, but remains dangerously sensitive to outliers. Hence, z-scores are more commonly used as they place all values within a range of [-3,3] and allow for the data to maintain the same distribution shape. For all continuous data in this set, $(X-\mu) /(\sigma)$ was applied for the normalization technique. In regards to the discrete/categorical data, one-hot encoding was used in order to convert attributes into binary vectors, meaning each attribute is assigned a value that is equidistant from every other value, such that the model does not weight them differently. By assigning each attribute a value of 0 or 1 in a binary vector, it is better suited for analysis as everything is equidistant from one another.

Decision Trees

Our decision tree implementation achieved an accuracy of 81.62% against the test imputed (sans oversampling). The results indicate more confidence in predicting the < 50k than in predicting the > 50k. So I would not use it to target high earners because it would yield.

```
 Method 1: Decision Tree Training & Evaluation
Target classes encoded: ['<=50K' '>50K']
Using External Test File for evaluation.
Training set size: 49440
Test set size: 16281

Model Accuracy: 0.8162

Classification Report:
              precision    recall  f1-score   support

       <=50K       0.88      0.88      0.88     12435
        >50K       0.61      0.62      0.61      3846

    accuracy                           0.82     16281
   macro avg       0.75      0.75      0.75     16281
weighted avg       0.82      0.82      0.82     16281

Confusion Matrix:
[[10922  1513]
 [ 1480  2366]]
Decision Tree Complete.
```

Confusion Matrix: Decision Tree Result

	Predicted: <=50K	Predicted: >50K
Actual: <=50K	10,922	1513
Actual: >50K	1,480	2,366

Logistic Regression

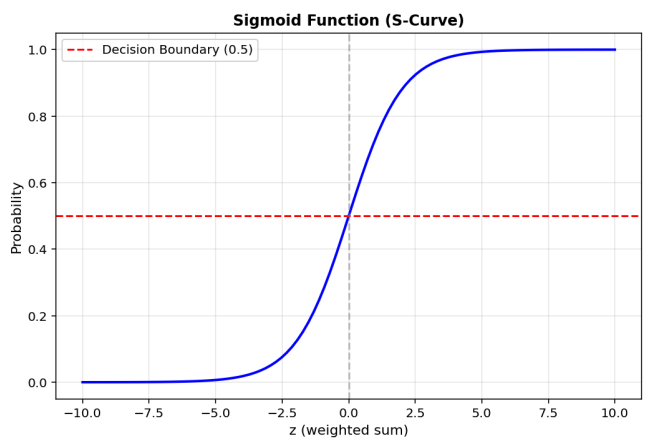
Model Structure and justification

Despite what its name indicates, this classification model opertates on the notion that it is finding a binary result; yes/no, true/false, healthy/sick. The core of the model is to be able to estimate what the probability is that an input will belong to class 0 or 1 (in this case, less than or equal to $50,000 or greater than that). Instead of using a distance formula to calculate a regression line, it uses a sigmoid function, which transforms given values to a range of [0,1], where closer to 1 means a large positive and vice versa for 0. For this model, only a couple of assumptions are made: observations are independent of one another, little to no multicollinearity,

and that there is a large sample size. This regression will produce two metrics in regards to the health of the core model: an S-Curve and a confusion matrix. An S-curve demonstrates how the model will predict between class 0 and class 1, meaning it shows the predicted probability as the input changes. A confusion matrix demonstrates four different values: True Positive (TP), True Negative (TN), False Negative (FN), and False Positive (FP). It will basically show the model's accuracy at predicting those four traits, along with three other key metrics (as listed below) which are precision, recall, and an F1 score. Precision will indicate how many positives the model actually produces, recall will explain how many of the actual positives the model got correct, and the F1 score is a harmonic mean between the two prior mentioned.

Model Evaluation

After running the regression, the data sets out to create the two following figures. The S-curve in this case is symmetric about the center line (0.5), in which case it is saying that the data being used is symmetric itself and the model has little to no bias in predicting class 0 or 1. This is key because if data were to not be equal, because this is dealing with a binary result, the curve would favor one over the other, leading to a precision of 100%, giving the user not scalable information that can be used for analysis.



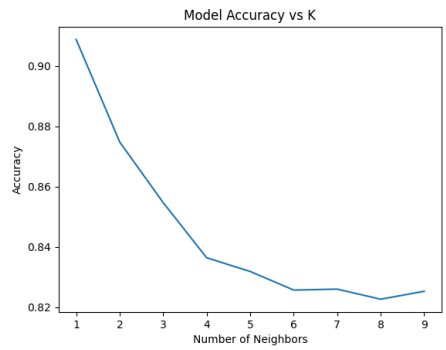

```
Model Performance Metrics

Accuracy: 80.58%

Precision: 55.86%

Recall: 84.74%

F1-Score: 67.33%

True Positives: 3259
True Negatives: 9860
False Positives: 2575
False Negatives: 587
```

The more important metrics are: 1) the accuracy of the model, 2) precision, 3) recall, and the 4) F1 score. The general accuracy of the model stands at 80.58%, which is within the general threshold of a 'good model.' However, there are definitely factors bringing it down, as the error is still ~20%, so the accuracy error is to be taken into account in regards to results provided by this model. A big driver of this decrease is a performance matrix labeled precision, which basically provides the ratio of all the positives, only 56% were actually shown as positive. Meaning it is really not that effective at predicting positive predictions on net. Finally, the F1 score is a balancing of the precision and recall (how many poisitives it can find and how good it is at detecting false positives), in which the score is sufficient for some practical use but also should be used with caution. It is important to reiterate that these are the results composed from balanced data using oversampling, in which the class imbalance would not result in the precision

being lower than usual. Furthermore, the symmetry being 0.5 proves that the algorithm is not inherently biased towards either class, as moving the center point +/- 0.1 would drop the accuracy by ~30% if not more, rendering the model useless in its entirety.

K-Nearest Neighborhood (KNN)

Starting off by taking the pre-processed dataset, we perform our data segmentation by dropping our true value Y (income) to assign it as our target, defining the remaining columns as our feature values. From here, we performed a 80/20 split on our dataset, creating an internal training and validation set to "build" our algorithm. These are labeled at X_train, y_train, X_test, and y_test. The last two "test" datasets are serving as our validation data. Following this we ran our KNeighborClassifer importing the scikit-learn package with our K value parameter, and our distance metric as the Euclidean Distance. Remembering that Euclidean distance is just the Minkowski distance, with the p = 2, creating our straight line distance metric.

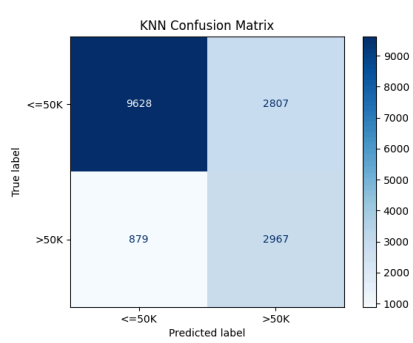


In choosing our value for K, we opted for "elbow finding". At first, we simply graphed model accuracy over a given range of k-values. As we increase our number of neighbors, we see that model accuracy only decreases until reaching a steady flatline around~82%. In deciding our k-value, we didn't want to lean too close to the lower values as accuracy increases rapidly making it extremely easy to move into the realm of overfitting. Conversely, since we can't use decimal values as K. Hence, we decided on the next readable and heuristic value of k = 5.

Prior to predicting target values, we must highlight the utilization of "knn.fit" in our model as it reveals an important feature about KNN, which is that it is a lazy learner. This means that contrary to all our other classification models – we're not actually building an algorithm here and our model isn't actually "learning" anything. KNN only operates for every new instance provided. "knn.fit" allows us to store the training data that is used to compare against the new given instance for every iteration we run it. Finally, we're able to generate predictions for Y utilizing our X value testing data. For each test point initiated from X_test, we calculate its distance from every point in X_train. It selects the K closest training points (lowest distances), and each neighbor "votes" on the class label using the majority vote model to determine the prediction for that test point. In the case of k = 1, our testing point simply aligns with the class value of the closest training point. Which is reflective of our high model accuracy output.

Analysis and Model Evaluation

Given that we've addressed normalization during pre-processing, we don't have significant outputs for feature distribution (e.g. a histogram). That being said, after running



the final census-income.test data on our KNN algorithm, we can now discuss the last two significant figures for analysis – Confusion Matrix and our Final Classification Report.

From our model application on the census-income.test data-set, we achieved an overall accuracy of 77.36%. However, using the classification report we can see our figures reveal a clear class imbalance, where <=50k (high income) has a significantly higher support value than >50k (low income). However when taking a look at our confusion matrix we see that our model classified the majority of the low income instances correctly staying consistent with its high precision. With 879 of those predictions being false positives. Conversely in our 3,846 high income records, only 2,967 of them were classified correctly. Our KNN model is even in recall performance (0.77); but precision performance has a clear discrepancy with performance falling short with high income records (0.51). This means that of the instances predicted to be this class, very few of them were the correct classification – reflecting the high number of false positives in our matrix. Our model thus tends to over-predict high income. It is clear the skewed distribution of our classes throughout the dataset influences the utilization of our model heavily. Since KNN uses the majority vote model among neighbors, this imbalance can explain the pattern of overall low performance in the higher income class. Overall, our model excels in finding the correct classes but falls short in predicting an underrepresented class.

```
Accuracy: 0.7736011301517106

Classification Report:

              precision    recall  f1-score   support

       <=50K       0.92      0.77      0.84     12435
        >50K       0.51      0.77      0.62      3846

    accuracy                           0.77     16281
   macro avg       0.72      0.77      0.73     16281
weighted avg       0.82      0.77      0.79     16281
```

Recommended practices for future utilization includes but is not limited to utilizing other methods of learning from unbalanced datasets to improve class balance and thus overall model performance.

Ensemble Method

For this project, we chose to use the Ensemble method to reduce overfitting of the base models we used and create a strong learner. An ensemble method is a collection of base learners, where each model learns the target function and combines its outputs for a final prediction. With the ensemble method, we attempt to smooth out any contrast between the single models we used. All three models are trained on the same z-scaled, one-hot encoded features.

Our base learners within the ensemble are our Decision Tree, Logistic Regression, and KNN model. By combining predictions using the ensemble method, any errors made by individual models tend to offset each other, while correct decisions are reinforced, leading to a more accurate overall prediction. Using the ensemble method also allows us to increase the robustness of our analysis, as ensemble methods are less sensitive to outliers and noisy data points. Each of these models produced an income class probability, and these probabilities were combined into a weighted average to obtain our ensemble prediction. We used a 50% threshold to assign class labels.

We also weighted each learner to best reflect the reliability of each model. To determine the weight to be assigned to each model, we calculated the relative accuracy of each model. For each model's accuracy, we divided its individual accuracy percentage by the sum of the accuracies of the three models. The weights for each model are outlined below. As is evident, they came out to be similar numbers ranging from 0.32 to 0.35 (summing to 1).

Learner	**Weight**
Decision Tree	0.32
Logistic Regression	0.33
KNN	0.35

With these weights, the final ensemble accuracy came out to be 82.3% when evaluated on the test set. This is a higher accuracy than any of the individual models. As compared to the results of the decision tree confusion matrix, the ensemble method used here did reduce the number of false negatives. The Decision Tree had 1,500 false negatives while the ensemble had 1,097, which is a significant reduction. We have outlined the final confusion matrix below.

Confusion Matrix - Ensemble Method

10651	1784
1097	2749

In terms of true values <=50K, there are 10,651 correct and 1,784 misclassified. For true values >50K, there are 2,749 correct and 1,097 misclassified. For the <=50K class, the ensemble had a precision of 0.91 and a recall of 0.86, which is a high performance on the majority class. The ensemble method has a recall of 0.71 for the >50K class and a precision of 0.61. This is higher than the Decision Tree recall (0.61), though lower than the Logistic Regression recall (0.85) and KNN (0.77). This shows that while the ensemble did a better job than the Decision Tree model of correctly identifying the >50K cases, Logistic Regression and KNN caught more positives than the ensemble method. However, it is important to note the tradeoffs. KNN and Logistic Regression had a lower precision (.51 and .56), so many of the predicted positives were actually false positives. Because the ensemble method has a precision of 0.61 for the >50K class, this means that while it did not catch as many positives as Logistic Regression and KNN, it made fewer false positive predictions. This is reflective of a more balanced trade-off between precision and recall compared to the individual models.

Conclusion

After synthesizing our findings we can support that machine learning classification models have the feasible capability to provide insight into income prediction with respect to attributes like education, occupation and demographics. Each model has their own strengths and weaknesses but as noted before, our ensemble method achieved a noticeable improvement in classification accuracy. Key takeaways include our ensemble approach to balance precision-recall tradeoff by reducing false negatives by ~27% compared to specifically decision trees alone (1,480 to 1,097). Furthermore, while we note class imbalance as a challenge in this dataset – it is still relatively reflective of current real-world economic disparities. Meaning for future improvements we can aim to possibly utilize datasets with broader feature sets, to possibly include regional statistics like cost-of-living and industry-wage trends. Additionally, one may look into more complex methods of balancing datasets, the lack of model bias may be able to produce results that can educate next steps in exploring this topic further.